

Back-end PPM 2026

Project Assignment: Full-Stack Web Application or REST API

Allowed frameworks: Django / Django REST Framework or Flask with equivalent requirements

Main objective

Develop a small but complete back-end application. Students must choose one of the proposed project tracks and deliver a working GitHub repository, a deployed version, a populated SQLite demo database, and clear instructions that allow the instructor to test the project immediately.

Email me for questions: simone.ricci@unifi.it

Submission Google Form: <https://forms.gle/SJE19bFLXxMTVd2EA>

Submission Deadline: **12/07/2026**

1. Submission rules

- Each project must be submitted through a dedicated GitHub repository.
- The repository must contain everything required to run the project locally.
- The project must be deployed on a free platform or on a platform with a free plan, such as Render, Railway, PythonAnywhere, Vercel, Heroku, or an equivalent service.
- The deployment URL must be clearly reported in the README.md file.
- The repository must include a small pre-populated SQLite database so that the application can be explored immediately.
- The README.md file must list the available demo accounts, including username, password, and role for each account.
- Do not use personal passwords, real accounts, private API keys, or sensitive data. Demo credentials must be created only for the evaluation of the project.

2. Choice of project type

Students must choose one of the two project types below. The two options are intended to be comparable in workload: the full-stack project requires a usable browser interface, while the REST API project requires a well-designed API, authentication, permissions, JSON validation, and complete testing instructions.

Project type	What must be delivered	Recommended when
Full-Stack Web Application	A complete web application with models, views, templates, forms, authentication, permissions, HTML/CSS interface, and navigable pages.	Choose this option if you want to build a site that can be used directly from the browser.
REST API	A back-end application with REST endpoints, token/JWT authentication, serializers or request validation, role-based permissions, JSON responses, and a documented testing workflow or minimal client.	Choose this option if you want to focus on API design and on the separation between back-end and client.

Balanced scope rule

A REST API project is not a reduced version of the full-stack project. It does not need a complex HTML interface, but it must compensate through stronger API documentation, endpoint coverage, request/response validation, authentication, role-based permissions, and reproducible testing commands.

3. Allowed frameworks and equivalent requirements

- **Django / Django REST Framework:** recommended because they are coherent with the course and provide built-in support for models, authentication, permissions, forms, serializers, and class-based views.
- **Flask:** allowed if the project satisfies equivalent requirements: modular structure, relational models, authentication, roles/permissions, input validation, SQLite persistence, deployment, and complete documentation.

4. Mandatory GitHub repository requirements

Item	Required	Description
README.md	Yes	Must explain the project goal, main features, installation, local execution, demo accounts, user roles, deployment link, and, for REST APIs, endpoint documentation and testing commands.
requirements.txt	Yes	Must list all Python dependencies required to run the project. Do not commit the virtual environment.
SQLite database	Yes	Include db.sqlite3 for Django or an equivalent SQLite file for Flask. The database must contain initial data and working demo accounts.
Source code	Yes	Code must be organized into coherent apps, blueprints, modules, or packages. Avoid unused files, duplicated code, and unfinished features.
Migrations	Yes for Django	Django migrations must be present and consistent with the models. The project should also be recreatable from an empty database.
Deployment	Yes	The deployed site or API must be reachable online through the link reported in README.md.
Testing client or testing workflow	Required for REST API	A simple HTML/JavaScript client, Python script, Postman/Insomnia collection, or a complete HTTPie-based workflow is acceptable if fully documented in README.md.

5. README.md: required structure

- Project title and student names.
- Chosen project type: Full-Stack Web Application or REST API.
- Framework used: Django, Django REST Framework, or Flask.
- Short description of the application and its purpose.
- Implemented features, preferably grouped by user role.
- Local installation instructions: clone repository, create virtual environment, install requirements, apply migrations if needed, and start the server.
- Name of the included SQLite database file and confirmation that it contains demo data.
- Demo accounts with username, password, and role.
- Online deployment link.
- For REST APIs: endpoint table with HTTP method, URL, authentication requirement, allowed role, request body, response example, and short description.
- For REST APIs using HTTPie: installation link, base URL, login command, token usage, and example commands for the main operations.

Example demo-account section

- admin_demo / admin12345 - administrator or superuser
- user_demo / user12345 - standard user
- manager_demo / manager12345 - advanced role, such as organizer, curator, coach, store manager, moderator, or premium user

6. Common technical requirements

Requirement	Full-Stack Web Application	REST API
Modular structure	At least 2 Django apps, or at least 2 Flask blueprints/modules.	At least 2 Django/DRF apps, or at least 2 Flask blueprints/modules.
Relational data model	At least 2 relations between tables, such as ForeignKey, Many-to-Many, or One-to-One.	At least 2 relations between API resources/tables, exposed through endpoints or nested representations when appropriate.
Custom user model or profile	Extend and customize the user model, for example through AbstractUser or a motivated profile model.	Define a customized user model/profile and expose authentication-related endpoints or documented login flow.
Roles and permissions	At least 2 groups/roles with different permissions, enforced in views and reflected in the interface.	At least 2 roles with permission checks enforced in endpoints, not only described in the README.
Class-based view	At least 1 generic class-based view or equivalent class-based controller.	At least 1 APIView, ViewSet, MethodView, Resource, or equivalent class-based endpoint.
Input validation	Use Form, ModelForm, or explicit validation with clear error messages.	Use serializers, schemas, validators, or explicit JSON validation with clear error responses.
Core CRUD logic	At least one main resource must support create, read, update, and delete operations according to permissions.	At least one main API resource must expose list/retrieve/create/update/delete operations according to permissions.
Demo data	SQLite database included and pre-populated with realistic demo data and accounts.	SQLite database included and pre-populated with realistic demo data, accounts, and API-testable records.
Deployment	A deployed web application with working login and navigable pages.	A deployed API with reachable endpoints and documented commands for testing it online.

7. Specific requirements for the Full-Stack Web Application

- Implement a complete website navigable from a browser.
- The site must include dynamic back-end functionality: users must be able to create, edit, view, or delete data according to their permissions.
- Use HTML templates and CSS. Frameworks such as Bootstrap are allowed.
- Implement login, logout, and, when useful for the project, user registration.
- Show different pages, actions, menus, buttons, or permissions depending on the user role.
- Include at least one class-based view or generic view.
- Include meaningful forms with validation and user-friendly error messages.
- Provide clear feedback when the user does not have permission to perform an action.
- The deployed version must allow the instructor to log in with the demo accounts and test the main workflow directly from the browser.

8. Specific requirements for the REST API

- Implement a REST API with documented JSON endpoints.
- Implement authentication through token, JWT, or an equivalent secure mechanism.
- Use role-based permissions in the API logic.
- Use HTTP methods correctly: GET for reading, POST for creation, PUT/PATCH for updates, and DELETE for deletion.
- Return coherent JSON responses and appropriate HTTP status codes.
- Validate request data and return clear JSON errors for invalid input.
- Expose a sufficient endpoint surface: authentication endpoints, list/detail endpoints, create/update/delete endpoints for at least one main resource, and at least one role-specific or action endpoint.
- Document all main endpoints in README.md, including method, URL, required authentication, allowed role, example request, and expected response.
- Include a testing workflow or minimal client. A complete HTTPie workflow is accepted if it is fully documented in README.md.

HTTPie is allowed for REST API testing

Students may use HTTPie (<https://httpie.io/>) as the required minimal REST API client/testing workflow. In that case, the README.md must include complete commands to authenticate, store or copy the token, call the main endpoints, test role-specific permissions, and reproduce the main project workflow on both local and deployed URLs.

9. Full-Stack Web Application tracks

Track	Minimum features	Minimum roles and permissions
1. E-commerce Store	Product catalogue, categories, product detail page, search/filter, shopping cart, order creation, and order management.	Customer: browse products, manage cart, and view own orders. Store manager: manage products, categories, and orders.
2. Social Network	User profiles, posts, friend/follow relationships or requests, feed/news page, and profile pages.	Standard user: manage own profile and posts. Moderator: delete inappropriate posts and deactivate or ban accounts.
3. Event Management System	Event creation, event detail page, event registration, attendee list, and registration management.	Attendee: view events and manage own registrations. Organizer: create/update/delete own events and view attendees.
4. Music Streaming Service	Song catalogue, genres, playlists, user profiles, search/filter, and simple recommendations or suggestions.	Listener: browse songs and manage own playlists. Curator: manage songs, genres, and the music library.
5. Fitness Tracking Application	Workout logs, goals, progress entries, user profile, history page, and simple charts or summaries.	Standard user: manage own workouts, progress entries, and goals. Coach: view assigned users and provide feedback.

10. REST API tracks

Each REST API track must include authentication, role-based permissions, data validation, documented endpoints, and a reproducible testing workflow. External paid services are not required: when a domain such as weather or stock data is chosen, data may be simulated or loaded from a local dataset, as long as the API design is coherent.

Track	Minimum endpoints/features	Minimum roles and permissions
1. Social Media API	Authentication, create/list/retrieve/update/delete posts, follow users, like posts, comments, and feed/timeline endpoint.	Authenticated user: manage own content and interactions. Moderator: remove inappropriate content and block accounts.
2. E-commerce API	Product list/detail, category endpoints, cart management, order creation, product availability, and basic order status management.	Customer: manage own cart and orders. Store manager: manage products, categories, and orders.
3. Weather Forecast API	Forecast query endpoint by location/date/time, daily request tracking, saved query history for premium users, and list/detail endpoints for saved queries.	Public/anonymous: limited number of forecast requests per day. Premium user: higher or unlimited limit and saved query history.
4. Ticket Reservation API	Event list/detail, seat availability, create/update/cancel reservations, and reservation status endpoint.	Anonymous: read-only access to events. Registered user: create, update, and cancel own reservations.
5. Stock Market API	Quote endpoint, asset list/detail, historical prices, portfolio creation/update, and portfolio valuation endpoint. Stock data may be simulated.	Basic user: manage portfolio and access quotes with a daily limit. Pro user: higher limit and extended historical data.
6. Polling Application API	Poll list/detail, create/update/delete own polls, vote endpoint, results endpoint, and optional closing/opening of polls.	Anonymous: list polls and view results. Authenticated user: create polls, vote, and manage own polls.

11. Required demo and testing material

- The included SQLite database must contain enough data to test the main workflows without manually creating everything from zero.
- There must be at least one demo account for each relevant role.
- For full-stack projects, the README.md should describe a short browser-based testing scenario: login, navigate to a feature, create/edit data, test permissions, and verify the result.
- For REST API projects, the README.md must include a command-based or client-based scenario: login, call public endpoint, call authenticated endpoint, create/update/delete data, and test at least one forbidden action with a user that lacks permissions.
- The deployed version should be consistent with the code in the submitted repository.